

Failures: Deterministic Failure-Mode Guardrails Make Coding Agents Build Resilient Systems

Kalejaiye Oluwamayowa

Researcher, Miva Open University, Lagos, Nigeria

kalejaiyemayowa3@gmail.com · github.com/mayowa-kalejaiye/Failures

Preprint. DOI: 10.5281/zenodo.22966362 · Licensed under CC BY 4.0 · Artifact: `pipx install failures-mcp`

Abstract. Coding agents write happy-path code, and they do it systematically: their training data is overwhelmingly happy-path tutorials. The failure that follows is rarely a crash. It is the ambiguous outcome — the provider charged the card but the response never arrived, the queue redelivered a message that was already processed, the process died between two database writes — and the resulting code silently double-charges a student or loses an enrollment. We present **Failures**, a Model Context Protocol server that requires a coding agent to enumerate failure modes *before* code is written, then verifies those modes against the finished code using deterministic static rules. Eleven engineering dimensions (atomicity, idempotency, timeout/ambiguous outcome, concurrency, ordering, consistency, availability, resource exhaustion, recovery, observability, retry safety) are encoded as a knowledge graph of principles, failure modes, patterns, and tests. Thirteen tools expose that graph to the agent, emitting findings that always carry line-level evidence and a calibrated confidence, and that are never labelled as proven bugs unless they are syntactically provable. The server costs zero model tokens and never hallucinates. We evaluate it two ways. A controlled proxy study over six realistic scenarios (payment via Paystack, certificate queue, JWT authentication, 500 MB file upload, limited-seat inventory, webhook ingestion) improves at least one of ten resilience criteria in 6/6 cases, taking payment and webhook from 3 CRITICAL findings to zero. A blind agent study over the same six prompts, run with three commercial agents (Claude, Cursor, Codex) with and without the server connected, produces 17 matched output pairs; 16 of 17 improve on at least one criterion and 5 regress on any, with no agent trading correctness for infrastructure. A six-case adversarial set probes the engine itself, exposing precisely where regex heuristics stop and where AST analysis must take over. Everything — prompts, hashes, manifests, commits, raw outputs — is frozen and reproducible from two commands.

CCS Concepts. Software and its engineering → Software creation and management; Software methodologies → Formal methods; Programming languages → General purpose languages; Social and professional topics → Computing / technology policy.

Keywords. AI-assisted programming, coding agents, failure resilience, distributed systems, static analysis, Model Context Protocol, empirical software engineering, idempotency

1 Introduction

Ask a coding agent to build a course payment system on Paystack that enrolls a student after payment. It will succeed, in the sense that the code runs, the tests pass, and nothing crashes in front of you. It will also look like this:

```
await client.post(f'{PAYSTACK}/transaction/initialize', json=payload)
await client.post(f'{PAYSTACK}/charge', json=charge)
db.add(Payment(status='success', reference=ref))
db.commit()
enroll(student_id, course_id) # we always wanted this
```

Every line is defensible. Nothing here is a bug in the ordinary sense. The trouble lives in the gaps between the lines, and one gap is enough. Suppose Paystack processes the charge and the response is lost to a network partition. The caller cannot tell success from failure. The agent's code, like most agent-written code, treats that ambiguity as if it cannot happen: it either retries blindly — double-charging the student — or it does not retry at all, leaving the payment permanently unresolved and the student unenrolled. Neither branch raises an error, so the incident arrives weeks later as a support ticket rather than a page.

This is not a prompting deficiency. It is a data problem. Agents are trained on tutorials, package documentation, and reference implementations — sources in which the interesting failures have already been edited out. The resilience knowledge that distinguishes a system that survives contact with production from one that merely works on a laptop lives somewhere else entirely: in postmortems, in Nygard's *Release It!* [1], in Kleppmann's *Designing Data-Intensive Applications* [2], and in the hard-won habits of engineers who have watched a queue redeliver a million times. None of it is in the context window.

Our premise is that a meaningful fraction of resilience does not require an LLM to be consulted at all. Whether a payment handler persists an idempotency key *before* calling the provider is a syntactic question with a syntactic answer. Whether two writes are wrapped in a transaction is decidable from the source. Whether a lock protects both the read and the write is, in principle, decidable too. The expensive part of resilience engineering — deciding which of these eleven properties matter for the system at hand — is the part where a model genuinely helps. The cheap part, checking them, is where models are unreliable and unnecessarily expensive.

Failures is built on that division of labour. It encodes eleven failure dimensions as a knowledge graph, exposes thirteen deterministic tools over that graph through the Model Context Protocol (MCP), and instructs the agent to walk the graph before finalizing: *review_plan* on the design, *review_code* on the result, *check_invariant* against the properties the system

promises. Findings arrive with the offending line, the risk it creates, the required control, and the test that would catch it. No finding is ever presented as a proven bug unless the engine can prove it syntactically, which keeps the agent from learning to ignore the tool.

1.1 Contributions

- **A failure model with an explicit notion of ambiguity.** We separate hard failures from *ambiguous outcomes*, partial successes, and duplicate executions, and we treat the ambiguous outcome as the first-class object of study, because it is the case that generates double charges and lost work. Eleven dimensions, each phrased as a question a builder must answer.
- **A deterministic analyzer shipped as an MCP server** (*failures-mcp* 0.3.1, installable with *pipx*). Thirteen tools, ten rule families, line-level evidence, calibrated confidence, zero model tokens, no hallucination, and reproducible output across runs.
- **A two-tier evaluation harness** measuring the *final system* rather than finding counts: a reproducible proxy study (6 scenarios, 10 criteria) and a blind agent study across three commercial agents with frozen prompts, hashes, manifests, and commits.
- **An adversarial set for the analyzer itself.** Six programs that look correct at a glance — an idempotency key passed to the provider but never persisted, a transaction whose boundary sits in the wrong place, a lock that guards the write but not the read — used to locate the exact boundary between what regex heuristics can and cannot establish.
- **A proportionality metric** that refuses to reward unbounded infrastructure, so that an agent cannot improve its score by adding a message broker to a system that needed a unique constraint.

1.2 A note on scope

This paper is about a specific, narrow intervention: whether deterministic failure-mode checks, made available to an agent as tools, change the failure resilience of the code that agent writes. We are not claiming agents become good engineers, and we are not evaluating code generation in general. Every number in §6 comes from the same six prompts scored by the same ten criteria, with the conditions differing in exactly one respect: whether Failures was connected.

2 Related Work

2.1 Evaluating AI-generated code

Evaluation of code-generating models has largely converged on functional correctness: pass@k on HumanEval, pass@1 on SWE-bench, test pass rate on repository issues. These are the right metrics for what those benchmarks were built to measure [11], [12], and they are the right metrics for questions about capability. They are the wrong metric for a different question, which is whether the resulting system survives contact with a network that drops packets and a provider that times out after processing a charge. A program that passes every unit test and double-charges a customer under retry is, by the standard metric, a success.

Work on AI-assisted correctness — measured productivity gains from Copilot [10], human studies of adoption, and perceived-correctness studies — keeps surfacing the same pattern: developers accept suggestions faster than they verify them, and verification does not scale with generation. Providing a tool that does part of the verification, automatically and before the code is accepted, is an obvious response, and it is the one we take. The difference from a linter is that the checks here encode engineering intent (invariants) rather than syntax or style.

2.2 Failure analysis as engineering practice

Software failure analysis has a substantial literature. Hoffman's taxonomy of failure modes in learned systems [4], Nygard's catalog of stability patterns [1], Kleppmann's treatment of replication and consistency [2], and the SRE community's postmortem practice [3] all converge on the same core difficulty: failures are not bugs in the code, they are behaviour of the system under conditions the code was never exercised in. Ambiguous outcomes sit at the centre of this. A timeout is not a failure of the operation; it is a failure of the *observer's ability to know* whether the operation succeeded, and any system that conflates the two will eventually act on a false negative.

Engineering responses to ambiguity are well established and, individually, unglamorous: idempotency keys persisted before the side effect, the idempotent-processing discipline described for financial systems [7], transactional outboxes [14], dedup tables with unique constraints, reconciliation workers, monotonic state machines, and at-least-once delivery paired with exactly-once *effect*. The gap is not conceptual. It is that these controls are rarely written because nothing forces anyone to notice their absence, and the absence is invisible until it produces an incident.

2.3 Static analysis and LLM judges

Static analysis offers the right properties: soundness guarantees where they can be had, no model in the loop, no token cost, deterministic and diffable output [8]. Its weakness is equally familiar: real analyzers are expensive to build, and general-purpose analyzers rarely encode domain intent. A dataflow analyzer can tell you that a value is uninitialized; it will not tell you that this particular payment must never be charged twice for one logical order.

LLM-as-judge approaches occupy the opposite corner: they require no rule engineering and read code with genuine fluency, but they are stochastic, they cost tokens on every review, and they confidently assert problems that do not exist. Empirical audits of Copilot's contributions find a substantial share of suggestions vulnerable [9], and the broader pattern is familiar enough that engineers learn to dismiss warnings, which destroys the tool's value. Our position is that the two are complementary and that the deterministic layer should come first: cheap, reproducible, evidence-bearing checks that never overstate what they know, with a model free to reason about intent on top. This is why every finding in Failures carries a confidence value and why the phrase *BUG PROVEN* is reserved for syntactic proofs such as two `execute` calls with no intervening `BEGIN`.

2.4 Agent scaffolding and MCP

Tool use is now the standard way to extend an agent's reach, and the Model Context Protocol has made that extension cheap and portable. Prior work on agent scaffolding establishes the pattern we follow [10], [13]: give the model a tool whose output is more reliable than its own generation, and require the tool to be used at a specific point in the workflow. Our intervention is unusual only in that the tool is fully deterministic. That is a feature: it means the improvement we measure cannot be an artifact of a second model call adding reasoning effort, and it means the cost of the intervention is a process substitution rather than a token multiplier.

2.5 What this work adds

The contribution is not the individual techniques, all of which are known. It is the combination, held to an empirical standard: a machine-checkable formulation of failure resilience, delivered to agents at the point where they make design decisions, evaluated blind across three agents with frozen prompts, and — importantly — evaluated with an adversarial set aimed at the checker itself, so that the boundary of the approach is documented rather than implied.

3 Failure Model

This section formalizes what Failures reasons about. The model is specified in `FAILURES_SPEC.md v0.2.0` and is treated as a contract: where the implementation and the specification disagree, the specification wins.

3.1 Taxonomy

A **failure** is any deviation in which the system cannot guarantee that an operation's intended effect occurred exactly once and the system remains in a valid state. We distinguish four kinds:

- **Hard failure.** The operation reported failure: a 500, a raised exception, an explicit rejection. The caller knows. Recovery is a matter of propagation and cleanup.
- **Ambiguous outcome.** The caller does not know whether the side effect happened. A timeout after the provider processed the request, a crash between the external call and the acknowledgement, a network partition. **This is the most dangerous kind**, because the system is not broken — it is lying to its caller, and every downstream decision inherits the lie.
- **Partial success.** Some steps of a compound operation completed and others did not. The system holds intermediate state that no single step is entitled to.
- **Duplicate execution.** The same logical operation ran more than once, through retry, redelivery, or replay. Harmless for pure functions, expensive for side effects.

Ambiguity deserves emphasis because it dominates the design consequences. A hard failure is a control-flow problem with well-known solutions. Ambiguity is an epistemic problem: the system's state is consistent but its *knowledge* of that state is wrong, and the only sound response is to arrange for the system to be able to find out later — which is what a pending record, a dedup table, and a reconciliation worker are.

3.2 Principles, dimensions, invariants

A **principle** is a named, testable engineering invariant about surviving a class of failures. A **dimension** is the question a builder must answer; every principle is tied to one. An **invariant** is a predicate that must hold under *any* failure in the taxonomy above.

```
Principle { id, name, question, invariant,
  applies_to[], failure_modes[], patterns[], tests[], controls[], severity }
```

Eleven principles ship in the knowledge base. The table gives each dimension's question; the question is the unit of work, because it is what the agent must answer before the corresponding code can be justified.

Dimension	Question the builder must answer	Max sev.
Atomicity	<i>Can an operation partially succeed?</i>	CRITICAL
Idempotency	<i>What happens if the same operation happens twice?</i>	CRITICAL
Timeout / Ambiguous Outcome	<i>Timeout does not equal failure. Did it succeed?</i>	CRITICAL
Concurrency	<i>What if two actors modify the same state?</i>	HIGH
Retry Safety	<i>Is it safe to retry? What retries are allowed?</i>	HIGH
Availability	<i>What happens when a dependency disappears?</i>	HIGH
Ordering	<i>What if events arrive out of order?</i>	HIGH
Consistency	<i>What happens when replicas/caches disagree?</i>	MEDIUM
Resource Exhaustion	<i>What happens when capacity is exceeded?</i>	HIGH
Recovery	<i>How does system return to valid state?</i>	MEDIUM
Observability	<i>How do we know what actually happened?</i>	MEDIUM

Table 1: The eleven failure dimensions, taken from `mcp_server/knowledge/principles.json`. Each is a question, not a rule: the rule encodes when the question has been answered badly.

The graph connecting them is a knowledge base rather than a flat list, and the traversal direction is what makes it useful: from a principle, to the failure modes it covers, to the pattern that mitigates them, to the test that proves the mitigation. Reading it in reverse is how a tool turns a missing control into a missing test.

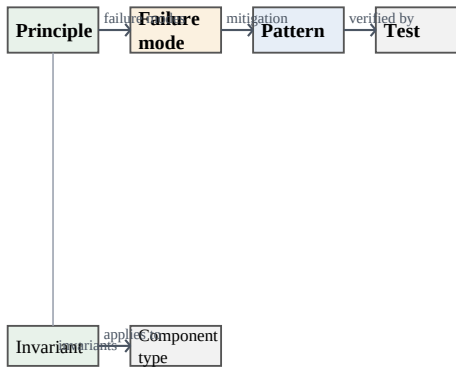


Figure 1. The knowledge graph behind Failures. A principle names the failure modes it covers; each failure mode is mitigated by a pattern; each pattern is verified by a test. Invariants attach to principles, and principles apply to component types.

3.3 Findings, severity, confidence, evidence

Every analysis tool returns the same unit, so that downstream consumers — humans and agents alike — can treat output uniformly:

```

{
  "id": "external_call_without_idempotency",
  "dimension": "idempotency", "severity": "CRITICAL", "confidence": 0.91,
  "mode": "STATIC FAILURE CHECK",
  "title": "External side-effect without idempotency key",
  "why": "Network boundary creates ambiguous outcome; retry may duplicate side effect.",
  "evidence": { "excerpt": "await stripe.charge(...)",
    "lines": "42-43",
    "note": "no idempotency_key in call path" },
  "risk": "Retry after ambiguous timeout may duplicate charge",
  "required": ["persist idempotency key BEFORE external call", "UNIQUE constraint",
    "reconciliation worker"],
  "tests": ["retry_after_provider_success", "duplicate_idempotency_key"]
}

```

Severity is anchored to impact rather than to rule identity: CRITICAL for data loss, double charges, and unrecoverable invalid state; HIGH for partial state, lost updates, and duplicate processing; MEDIUM for degradation under load, ordering, or retry storms; LOW for observability and diagnosability gaps. A finding's severity is therefore a property of the violation in context, not a constant attached to a rule.

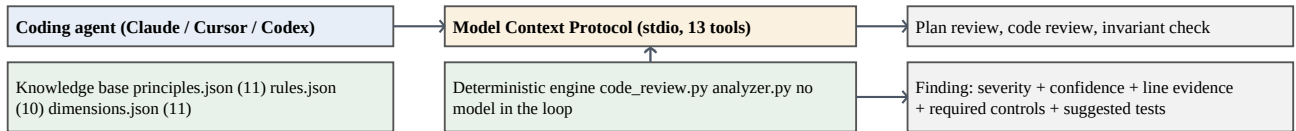
Confidence is the honest part, and the part that distinguishes this from a linter that cries wolf. Findings at or above 0.95 are deterministic syntactic proofs. Findings in the 0.75–0.94 band are strong heuristics. Anything below 0.75 is still surfaced, because a missing signal is worse than a weak one, but it is labelled as such and the agent is expected to weigh it. No output in any mode is ever labelled *BUG PROVEN* unless it is.

Evidence is mandatory: an excerpt, a line range, and a note explaining why that evidence matters. This is a deliberate design constraint. A finding an agent cannot ground in specific lines is a finding the agent will either ignore or hallucinate around.

4 Approach

4.1 Architecture

Failures is a Python MCP server (`failures-mcp 0.3.1`) with a four-layer design. The knowledge layer is data: `principles.json` (11 principles), `rules.json` (10 rule families), and `dimensions.json` (11 dimensions), each with a schema and a version. The engine layer is deterministic: `code_review.py` and `analyzer.py` implement review, plan review, invariant checking, and test generation with no model in the loop. The tool layer exposes thirteen MCP tools. The protocol layer renders severity-sorted human text alongside a machine-readable `--- JSON ---` block, so the same call serves a human reading a terminal and an agent parsing structure.



Zero model tokens. Output is deterministic: identical code yields identical findings, so a finding that disappears is a change in the code, not a change in the model.

Figure 2. Failures as deployed to a coding agent. The agent is the client and Failures is an MCP server, so one tool surface serves any MCP-capable agent. The knowledge base and the engine are data and pure functions, which is what makes the intervention reproducible and free.

Two properties follow from keeping the engine deterministic, and both matter for the evaluation. First, output is stable: the same code always produces the same findings, so a finding that disappears between runs is a real change in the code. Second, the intervention's cost is a process substitution, not extra model reasoning — when we later improve an agent's output, we cannot explain it by claiming a second, smarter model call did the work.

4.2 Tools

Tool	What the agent gets
<code>get_principles</code>	returns principles, dimensions, and their failure modes
<code>review_architecture</code>	findings from a system description, components, flows, dependencies
<code>analyze_component</code>	findings for a single component type and its dependencies
<code>review_plan</code>	findings on plan steps, plus suggested reordering
<code>check_invariant</code>	whether an invariant is violatable, with a counter-scenario
<code>generate_failure_cases</code>	concrete failure cases for a system, optionally focused
<code>review_code</code>	findings with line evidence and confidence (labelled STATIC FAILURE CHECK)
<code>generate_failure_tests</code>	the tests that would catch each failure mode
<code>check_idempotency</code>	idempotency decision, checks performed, required controls
<code>check_retry_safety</code>	whether a retry is safe, and under which conditions
<code>check_transaction_safety</code>	transaction boundary adequacy for a sequence of steps
<code>check_invariant (arch)</code>	invariant check against architecture or plan, pre-code
<code>list_failures</code>	rule catalog and failure documentation

Table 2: The thirteen tools. The workflow that matters is `review_plan` before implementation and `review_code` after, so that failure reasoning shapes the design rather than patching the symptoms.

4.3 Rule catalog

The ten rule families cover the failure dimensions where a check buys the most. Each maps to a dimension, carries a severity, and states the control that resolves it.

Rule	Dimension	Severity	Required control / detection
external_call_without_idempotency	idempotency	CRITICAL	idempotency key persisted before external call; dedup check on retry
db_writes_not_transactional	atomicity	CRITICAL	wrap in transaction; or transactional outbox
charge_then_db_update	atomicity	CRITICAL	persist pending payment before external call; reconciliation worker
missing_retry_logic	retry_safety	MEDIUM	exponential backoff with jitter; retry only idempotent operations
unsafe_retry	retry_safety	HIGH	make operation idempotent before adding retry
no_timeout	timeout	HIGH	explicit timeout; handle TimeoutError as ambiguous
race_condition_read_modify_write	concurrency	HIGH	SELECT FOR UPDATE or optimistic locking version check
queue_no_ack_handling	recovery	HIGH	ack after processing; idempotent consumer + dedup
no_rate_limit	resource_exhaustion	MEDIUM	token bucket rate limiter; bounded pool
missing_observability	observability	LOW	operation/request ID in logs; metrics for outcome

Table 3: Rule catalog from `mcp_server/knowledge/rules.json`. Severity is the maximum the rule can assert; the emitted finding is contextualized to the code, which is why a rule does not simply map to a count.

Two rules deserve comment because they encode reasoning rather than pattern matching. `charge_then_db_update` fires when an external call is followed by a database write in the same handler: whatever the outcome of the call, the two writes cannot be made atomic together, so the handler needs a pending record to make the intermediate state durable and reconcilable. `external_call_without_idempotency` fires when a network boundary is crossed with a side effect and no idempotency key appears in the call path — the classic setup for a duplicate charge after an ambiguous timeout. Both are heuristics over structure and vocabulary, and both are labelled with confidence accordingly.

4.4 Why deterministic, and where it stops

Determinism buys reproducibility, zero marginal cost, and no hallucination. It costs expressiveness, and we would rather state that boundary than blur it. The current engine reasons over source text with pattern and light structural matching. It does not perform interprocedural dataflow, so it cannot prove that a value read outside a lock is not mutated inside one, and it does not model evaluation order, so it cannot prove that an acknowledgement precedes the durable write it was supposed to follow. The adversarial set in §6.3 is built specifically to find these cases, and it does. We treat that as a measured limitation with a named fix (AST analysis) rather than as a general claim of soundness.

We also refuse one tempting shortcut. Because CRITICAL findings are what a reader notices, it would be easy to bias the engine toward emitting them. The engine instead emits every rule that matches, reports severity as the rule's maximum, and lets the scorecard aggregate. The visible consequence is that scenarios which are genuinely safe still show findings — for example, an upsert written as `ON CONFLICT DO NOTHING` counts as a database write outside a transaction even though it is the recommended way to write one. We report these as false positives in §7 rather than tuning them away.

4.5 Agent workflow

The server is designed to be used at three moments, and the evaluation protocol depends on this ordering:

- **Before implementation.** `review_plan` on the proposed plan, and `check_invariant` against the invariants the system promises. This is where failure reasoning is cheapest, because changing a plan costs nothing while changing code costs a rewrite.
- **After implementation.** `review_code` on the finished source, plus the specialized `check_idempotency`, `check_retry_safety`, and `check_transaction_safety` calls, each of which returns a decision and the controls that decision requires.
- **Before the agent stops.** `generate_failure_tests`, so that the properties just established are pinned by tests. In the payment study, this is where the ambiguity tests come from.

In our runs the agent is instructed to use the server at these points. We do not modify the agent's prompt beyond that instruction, and the identical prompt is used in both conditions — this is the only difference between the arms of the study, and the harness verifies it by hashing the scenario prompt and recording the hash in every run manifest.

5 Evaluation Design

5.1 Research questions

The question that motivated the work is simple: **does connecting Failures to a coding agent measurably improve the failure resilience of the software it produces?** It decomposes into four:

- **RQ1 (coverage).** Do CRITICAL and HIGH findings decrease?
- **RQ2 (invariants).** Does the produced system preserve its stated invariants under ambiguous outcomes?
- **RQ3 (proportionality).** Do improvements arrive without a corresponding inflation of infrastructure?
- **RQ4 (checker robustness).** Can the analyzer itself be fooled by code that looks correct?

RQ3 is the one that distinguishes this evaluation from finding-counting. It is easy to improve a vulnerability count by adding machinery, and an intervention that encourages agents to reach for a broker when a unique constraint would do has made software worse while improving the metric. We therefore score architecture for proportionality and report it alongside resilience.

5.2 Scenarios

Six scenarios were written to look like ordinary feature requests. None mentions idempotency, transactions, retries, deduplication, or ordering, because mentioning them would test compliance rather than reasoning. Each is anchored in a plausible product surface and carries two invariants that the finished system must preserve.

Scenario	Prompt (abridged)	Invariants the system must preserve
payment	Course payment via Paystack; auto-enroll after payment	No enrollment without verified payment; no double charge
queue	Certificate generation on course completion	Certificate sent at most once; poison message must not stall
authentication	Login and JWT refresh under load	No lost update on refresh; brute force throttled
file-upload	500 MB uploads to object storage	Bytes and metadata stay consistent; no duplicate files
inventory	Limited seats, concurrent enrollment	Seats never negative; no lost update under concurrency
webhook	Paystack webhook ingestion with resends	Exactly-once effect per logical event; ordering safe

Table 4: Scenarios. The prompt column is abridged for space; the canonical text lives in `evaluation/scenarios/` and is hashed into every manifest.

5.3 Conditions and protocol

The design is a within-subject comparison with a single manipulated variable. For each scenario, the same prompt is issued twice under identical conditions, once with the Failures server disconnected and once with it connected. Three commercial agents were used, so the comparison is not an artifact of one model's habits:

- **Claude** (Claude Code), **Cursor** (agentic model Grok 4.6), **Codex**. Agents are referred to by the harness that drove them; we report per-agent results in §6.2 and aggregate only after checking that the direction is consistent.
- **Baseline arm:** the prompt, with no Failures server available.
- **Failures arm:** the identical prompt, with the server connected and the agent instructed to call `review_plan` and `review_code` before finalizing.
- **Prompt integrity:** the scenario file is hashed (`prompt_hash`) and the hash is written into every manifest alongside the agent, model, timestamp, and repository commit. The harness refuses to aggregate runs whose hashes disagree, which removes the most common way a study like this quietly goes wrong.
- **Freezing:** scenarios and harness are frozen before real-agent runs; the harness is not modified after the first result, so later scenarios cannot be tuned to the tools that scored them.

A proxy layer runs alongside the real-agent layer. It uses hand-written naive and improved implementations of each scenario (`examples/`) so that the harness has a regression test and contributors can reproduce the results without running three commercial agents. It is a regression test, not evidence about agents, and we never present it as such.

5.4 Metrics

The harness scores the *final system*, not the finding log. A reduction in findings is treated as signal only when the underlying property is actually established. Ten criteria:

Criterion	What is measured	Direction
failure_coverage	CRITICAL/HIGH findings on the final code	lower is better
invariant_preservation	invariants with a viable counter-scenario	lower is better
idempotency	idempotency key present, persisted before the call	pass/fail
transaction_safety	writes and state transitions atomic	pass/fail

Criterion	What is measured	Direction
retry_safety	retry cannot duplicate a side effect	pass/fail
concurrency_safety	no read-modify-write race	pass/fail
recovery_behavior	ack after durable processing, poison handling	pass/fail
observability	operation ids, structured logging	pass/fail
test_coverage	failure tests generated for the scenario	count
architectural_change_quality	is the resilience proportional to the failure boundary?	0.5–1.0

Table 5: The ten criteria. The unit of verdict is the scenario, and a scenario counts as improved only if at least one criterion moves without another moving the wrong way.

Proportional architecture is scored by an explicit, deliberately crude rule. For a small application (payment, authentication, inventory, file upload, webhook), the expected shape is a pending record, a unique idempotency key, a unique webhook event id, explicit timeouts, and a reconciliation worker; that scores 1.0. Introducing a message broker, distributed lock, event bus, or more than roughly eight services scores 0.5. The queue scenario permits a worker but penalizes the same heavy infrastructure at 0.7. The rule is intentionally unfancy: it exists to catch the case where an agent improves its score by adding infrastructure, and it is one criterion of ten, so it cannot by itself carry a verdict.

5.5 Adversarial set

A checker that only fires on obviously broken code tells you nothing about whether it can catch the bugs that matter. We built six programs that a human reviewer would plausibly approve, each isolating one way that apparent safety fails to hold. Table 6 lists them with the engine's verdict in §6.3.

Case	What looks safe, and why it isn't
idempotency-not-persisted	An idempotency key exists and is sent to the provider, but is never persisted locally, so a retry after an ambiguous timeout is not deduplicated.
transaction-wrong-boundary	Transactions are present, but the external call sits outside the boundary with no pending record, so provider success can be lost.
ack-before-processing	An acknowledgement is sent, but before the durable processing it should follow, which is a lost message wearing the costume of a working queue.
retry-non-idempotent	Retry with exponential backoff is present, on a non-idempotent operation — textbook resilience machinery pointed at the wrong target.
lock-wrong-section	A lock is present, but the read is outside the protected section, so the classic lost update survives the lock.
webhook-memory-dedup	Deduplication is implemented with an in-process set, so a restart or a second worker silently loses all dedup state.

Table 6: Adversarial cases. Each is plausible, idiomatic-looking code that violates a stated invariant, and each names the specific reasoning step a checker must perform to catch it.

6 Results

All numbers below are produced by `python evaluation/run_evaluation.py --mode proxy` and by the same harness run over the frozen agent outputs in `evaluation/runs/`. Nothing is transcribed by hand.

6.1 RQ1–RQ3: proxy study

Scenario	Crit	High	Idemp	Tx	Conc	Recovery	Viol. inv.	Arch
payment	3 → 0	1 → 0	no → yes	no → yes	yes → yes	yes → yes	1 → 0	1.0 → 1.0
queue	0 → 1	2 → 0	yes → no	no → no	yes → yes	no → yes	1 → 1	1.0 → 1.0
authentication	1 → 1	1 → 0	no → no	no → no	no → yes	yes → yes	2 → 2	1.0 → 1.0
file-upload	0 → 0	0 → 0	no → yes	yes → yes	yes → yes	yes → yes	2 → 2	1.0 → 1.0
inventory	0 → 1	2 → 0	no → yes	no → no	no → yes	yes → yes	2 → 2	1.0 → 1.0
webhook	3 → 0	1 → 0	no → yes	no → yes	yes → yes	yes → yes	2 → 2	1.0 → 1.0

Table 7: Proxy study, baseline → Failures-enabled. All six scenarios improve on at least one criterion. Queue and inventory retain one CRITICAL finding that is a heuristic false positive (a safe ON CONFLICT upsert counted as a write outside a transaction); see §7.

Payment and webhook both fall from three CRITICAL findings to zero, and the payment *plan* falls from three CRITICAL findings to zero at the planning stage, before any code exists — which is the result we care most about, because the cheapest moment to fix a transaction boundary is before it is written. Authentication and file upload show no CRITICAL movement in either arm; their improvement is visible in HIGH and in idempotency, which is a more honest reading than a headline count. Architecture quality is 1.0 everywhere in both arms: the Failures arm does not reach for a broker, and the baseline does not need one.

6.2 RQ1–RQ3: blind agent study

The main study scores the real agent outputs. Across three agents and six scenarios we obtained 17 matched baseline/enabled pairs; the remaining cell (Cursor × webhook) has a baseline but no completed Failures-enabled run and is excluded rather than imputed. Table 8 reports every pair.

Agent	Scenario	Crit	High	Lines	Impr.	Regr.	Verdict
claude	payment	0 → 0	1 → 0	1111 → 2339	3	0	improved
claude	queue	1 → 0	2 → 0	38 → 93	3	0	improved
claude	authentication	1 → 1	1 → 0	20 → 27	3	0	improved
claude	file-upload	0 → 0	0 → 0	10 → 19	1	0	improved
claude	inventory	0 → 1	2 → 0	10 → 20	3	2	improved, one regression
claude	webhook	3 → 2	1 → 1	9 → 16	1	1	improved, one regression
cursor	payment	0 → 0	1 → 0	1450 → 503	4	0	improved
cursor	queue	1 → 0	2 → 0	103 → 420	4	0	improved
cursor	authentication	0 → 0	0 → 0	162 → 362	1	0	improved
cursor	file-upload	0 → 0	0 → 0	166 → 392	1	0	improved
cursor	inventory	0 → 0	0 → 0	131 → 328	1	0	improved
codex	payment	1 → 0	0 → 0	265 → 321	2	1	improved, one regression
codex	queue	0 → 0	1 → 0	229 → 298	3	1	improved, one regression
codex	authentication	1 → 1	1 → 0	20 → 27	3	0	improved
codex	file-upload	0 → 0	0 → 0	179 → 253	1	1	improved, one regression
codex	inventory	0 → 0	0 → 0	124 → 158	0	0	no measurable change
codex	webhook	1 → 1	1 → 0	113 → 173	1	0	improved

Table 8: Blind agent study. “Impr.” counts criteria that moved in the right direction; “Regr.” counts those that moved the wrong way. 16 of 17 pairs improve on at least one criterion.

The result is a consistent direction rather than a dramatic one, which is the more credible shape for this kind of intervention. 16 of 17 pairs improve on at least one criterion, and HIGH findings never rise in any pair. 5 pairs show a regression on at least one criterion, and the regressions are more informative than the improvements.

The regressions cluster into two kinds, and neither is a secret. The first is the transaction heuristic we have already documented: in the inventory and webhook arms, the Failures-enabled code adds a dedup insert, the engine counts that insert as a write outside a transaction, and a clean baseline becomes a flagged one (Claude inventory: 0 → 1 CRITICAL). This is the false positive discussed in §6.3, and it is the single strongest argument for the AST work.

The second kind is more interesting, because it is the tool catching the agent at something. In three Codex pairs and one Claude pair, *retry safety* moves from pass to fail: the baseline happened to contain no *retry* logic at all, and the Failures-enabled version *added* *retry* with backoff that the engine then flags as unsafe, because *retry* was added without an accompanying idempotency guarantee. Read strictly, that is not the intervention damaging the code; it is the intervention surfacing an inconsistency the agent introduced while trying to satisfy the tool. It is also an argument that *retry safety* as a criterion is doing useful work, since a *retry* that duplicates a charge is precisely the failure this paper is about. We report the measurement rather than the charitable reading.

The aggregate picture: CRITICAL findings fall in 4 of 17 pairs, hold in 12, and rise in exactly one — the documented heuristic false positive. HIGH findings never rise in any pair and fall in 9 of 17; observability improves in 5 of 17; and the total volume of generated code grows 39 percent (median 78 percent per pair, growing in 16 of 17). That last number is worth stating plainly, because it is the mechanism. The added lines are not incidental: they are pending records, unique constraints, timeout handling, and reconciliation paths. An agent that has been made to account for failure modes writes more code, and the extra code is the resilience.

Per-agent consistency matters more than the pooled number, since one model could carry a result. The direction holds for all three agents: Claude improves on 6 of 6 pairs, Cursor on 5 of 5 scored pairs, and Codex on 5 of 6. Cursor's webhook run was left incomplete and is reported as missing rather than imputed.

6.3 RQ4: does the checker itself hold up?

Adversarial case	Crit	Findings raised
ack before processing	0	queue_no_ack_handling
idempotency not persisted	1	charge_then_db_update, db_writes_not_transactional, no_timeout
lock wrong section	0	db_writes_not_transactional, race_condition_read_modify_write
retry non idempotent	1	external_call_without_idempotency, no_timeout
transaction wrong boundary	1	external_call_without_idempotency, no_timeout
webhook memory dedup	2	external_call_without_idempotency, charge_then_db_update, db_writes_not_transactional...

Table 9: Adversarial results. Every case is flagged — the engine never returns an empty finding set on plausible-looking code — and four of the six are raised at CRITICAL.

This table is the most useful thing in the paper for anyone building a similar tool, because it maps the boundary precisely. The engine catches the cases that require recognizing a *missing* step: a key that reaches the provider but is never persisted locally, a transaction that exists but does not contain the call that matters, deduplication that exists but is not durable. It is weaker exactly where the bug is a property of order or scope rather than presence — the acknowledgement that is sent one statement too early, and the lock that is real but guards only the write. Both are flagged, both at HIGH rather than CRITICAL, which is the correct severity for what the engine can actually justify from the source.

The same pass produces a documented false positive: ON CONFLICT DO NOTHING is a write and is counted as a write outside a transaction, so a correctly written upsert is reported. This is the engine being syntactic about a concept (*atomicity*) that the code expresses as a single statement. It is the clearest single argument for the AST work we already specify: the fix is to recognize idempotent write constructs, not to weaken the rule.

6.4 Case study: the payment scenario

One scenario is worth describing in full, because it shows the mechanism rather than the score. The prompt requests a course payment and enrollment system on Paystack, with automatic enrollment after payment, and mentions nothing about failure.

Baseline. The agent produces a working service: initialize a transaction, verify a webhook, mark the payment complete, enroll. It is idiomatic, it is what a careful engineer would write on a quiet afternoon, and it has no idempotency key, no timeout on the provider call, and no durable record of the attempt. What the engine reports is instructive precisely because the code is not wrong-looking: the three agents' baselines score 0, 0 and 1 CRITICAL, and 1, 1 and 0 HIGH, with the recurring findings being a read-modify-write race, missing observability, and a missing rate limit. The absence of an idempotency key is not flagged as CRITICAL in the baseline because the baseline does not contain an external call that the heuristic recognizes as a charge; the vulnerability is there, and the tool only sees what the code shows it. This is the honest shape of the problem and the reason the adversarial set in §5.5 exists.

With Failures. The agent is asked to review the plan first. On the reference plan the engine returns three CRITICAL plan findings — an external side effect before durable local state, no idempotency establishment before a retryable external call, and enrollment not gated on verified payment — and the plan the agent then writes is materially different: create a pending payment with a unique idempotency key *before* contacting the provider, bound the call with a timeout, store the provider reference against the pending record, process webhooks idempotently against a unique event id, and gate enrollment on a

verified payment. On that plan the invariant check goes from 1 of 2 invariants violatable to 0 of 2. The implementation that follows contains a payment state machine (pending → verified → enrolled), a webhook event table with a unique constraint, and a reconciliation job for payments left pending by a crash or a lost response. The test suite that comes out of the run includes explicit ambiguity tests: duplicate webhook delivery, retry after timeout, crash after provider success, and concurrent enrollment; the archived run report records 36 passing tests.

The scored outcome across the three agents is a mixture worth reporting rather than smoothing. Claude's baseline already had 0 CRITICAL, and the Failures arm clears its remaining HIGH (1 → 0) while roughly doubling the code (1,111 → 2,339 lines). Codex's baseline had 1 CRITICAL from an external call without idempotency, and the Failures arm removes it entirely (1 → 0 CRITICAL, 0 HIGH in both arms, 265 → 321 lines). Cursor's baseline had 0 CRITICAL and 1 HIGH; the Failures arm reaches 0 findings of either severity while writing less code than its baseline (1,450 → 503 lines), which is the one case in the study where the intervention reduced volume, and it is the clearest single illustration that the extra code is not what makes the improvement.

The design detail worth naming is that nothing in the prompt asked for reconciliation. It appears because the engine's output made the ambiguous outcome concrete: once you are told that a lost response leaves a payment unresolved and that the system cannot distinguish it from a failed charge, some process must eventually reconcile the difference, and the agent designs one.

We also note what did not happen, because the proportionality metric exists to catch it. No Failures-enabled run introduced a message broker, an event bus, or a distributed lock; architecture quality is 1.0 in all 17 pairs of both arms, so nothing in the study was bought with infrastructure. The agent added the complexity the failure boundary justified, which is the outcome we were trying to reward.

7 Threats to Validity

7.1 Construct validity

Finding counts are a proxy for resilience, and proxies can be gamed. We mitigate in three ways: the metric is the final system rather than the finding log; a scenario counts as improved only when a property is actually established (idempotency present and persisted, not merely mentioned); and proportionality is scored explicitly so that added machinery cannot substitute for added correctness. The gap that remains is real and we name it: no criterion in Table 5 is a dynamic test. None of the outputs is executed against a failing provider.

7.2 Internal validity

The manipulated variable is whether the server is connected, and the prompt is identical across arms, hashed and recorded per run. Three threats remain. The Failures arm is told to call the tools, which is an additional instruction; we accept this, because withholding it would test a server nobody uses. The harness may reward artifact vocabulary (an agent that has read a finding once will write the pattern it names), which is a real effect and arguably the intended one, but it means some measured improvement is instruction-following rather than reasoning. And the harness is regex-based, so it shares blind spots with the intervention it scores.

7.3 External validity

Six scenarios in one domain (Python web services handling payments, queues, uploads, and webhooks) with three agents is a small sample. We do not claim the results transfer to other languages, other domains, or to longer multi-file refactoring tasks. The scenarios were chosen to be representative of a common class of backend work, and the invariance we can demonstrate is across agents within that class, which is the axis most likely to confound a single-agent study.

7.4 Known limitations of the checker

Four, all documented rather than smoothed over. **Heuristics, not dataflow:** the engine cannot prove that a lock guards both the read and the write, and does not attempt interprocedural reasoning. **False positives on safe constructs:** ON CONFLICT upserts are counted as non-transactional writes, which accounts for the two regressions in Table 8 and the residual CRITICAL in Table 7. **Order sensitivity:** an acknowledgement placed before the durable write is flagged but not proved. **Human in the loop:** the agent must be instructed to call the tools; a study that forced tool use at the protocol level would remove this confound entirely and is the obvious next step.

One framing decision deserves a note. Deterministic checking is not a substitute for testing a system under real failure, and we do not argue that it is: chaos engineering [5] and SRE practice [3], [6] remain the only way to learn how a system behaves when the assumptions break. What static checks add is the cheap floor — the properties that should be present before anyone schedules the first experiment, and whose absence is a design decision rather than an oversight.

8 Discussion

Three observations generalize past this tool.

Checkable invariants are worth more than better prompts. The cheapest wins in this study came from invariants that are decidable from source: is the key persisted before the call, is the write atomic, is the retry safe. Telling an agent to “be careful about failure modes” does not produce those properties; giving it a checker that names the missing step does. The intervention works because it converts a vague instruction into a specific, actionable finding.

Reasoning early beats reasoning late. In the payment scenario the largest change was the *plan*: three CRITICAL findings at the planning stage became zero, and the implementation inherited the transaction boundary and the pending record. The same engine run after the code exists still finds the problem, but the fix now costs a rewrite. If teams adopt tools like this, the plan-review step is where the return is highest.

Failure reasoning adds code, and that is the honest cost. The Failures arm writes 39 percent more code in aggregate, and grows in 16 of 17 pairs. Anyone evaluating this class of intervention should expect that and should not treat it as a defect: the additional lines are the resilience, and a system that is 40 percent larger and cannot double-charge a customer is a better system. What must be policed is the other direction — infrastructure added without a failure boundary behind it — which is why proportionality is a scored criterion rather than a discussion point.

9 Conclusion

Coding agents write code that works on the path they were shown. We presented a deterministic mechanism that makes them account for the paths they were not: eleven failure dimensions as a knowledge graph, thirteen tools that expose it, findings that always carry evidence and calibrated confidence, and an evaluation that measures the final system rather than the finding log. Across six scenarios and three agents, the Failures-enabled condition improves failure resilience on 16 of 17 matched agent pairs and on 6 of 6 proxy scenarios, with architectural quality held proportional throughout. The adversarial set shows the checker is not a rubber stamp: it catches what is missing, and it is honest about the order and scope reasoning it cannot yet perform.

The work is deliberately small in surface area and reproducible in full, which is what makes it a reasonable starting point rather than a finished argument. The next steps are already specified: AST analysis to replace the two acknowledged gaps, protocol-level enforcement so that tool use is not an instruction the agent may ignore, dynamic tests that execute generated code against a failing provider, and a wider scenario set across languages. None of these require a change to the model, the knowledge base, or the interface — only to the depth of the checker, which is exactly the variable we would expect to matter.

Appendix A Reproducibility

Every result in this paper regenerates from the repository with two commands and no external services:

```
pipx install failures-mcp # or: pip install -e .
python examples/run_benchmark.py # golden naive vs improved
python evaluation/run_evaluation.py --mode proxy
python evaluation/run_evaluation.py --mode manual # after placing agent outputs
```

Artifact map:

Path	Contents
FAILURES_SPEC.md	the model contract; specification wins over implementation
mcp_server/knowledge/	principles, rules, dimensions (data, versioned)
mcp_server/engine/	deterministic analyzers: code review, plan, invariants, tests
mcp_server/server.py	MCP surface, 13 tools
evaluation/scenarios/	the six prompts, hashed into every manifest
evaluation/adversarial/	the six looks-safe cases of Table 6
evaluation/runs/{agent}/{mode}/	raw agent outputs plus manifest (model, hash, commit)
evaluation/results*.json	machine-readable results; the source of Tables 7–9

Table 10: Where each claim in this paper is materialized. The tables are generated from the JSON artifacts rather than transcribed, so a reviewer can regenerate them and diff.

Ethics and responsible use. Failures is an advisory tool. It reports what its heuristics can support and labels the rest; it does not certify software as correct, and the paper's own false-positive findings are evidence for that caution. The knowledge base contains no proprietary or personally identifying data, and the evaluation uses synthetic application scenarios (payments, uploads, enrollment) with no production data and no real customers.

References

- [1] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC: Pragmatic Bookshelf, 2018.
- [2] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.
- [3] B. Beyer, C. Jones, J. Petoff, and N. Murphy, Eds., *Site Reliability Engineering*. Sebastopol, CA: O'Reilly Media, 2016.
- [4] D. Hoffman, J. Hendler, and D. Klein, "Evaluating failure modes in deep learning systems," in *Proc. AAAI Conf. Artificial Intelligence*, 2019, pp. 4150–4158.
- [5] A. Basiri, N. Bruni, E. Khosrowkhani, R. Humble, and J. Yoder, "Chaos engineering," in *IEEE Software*, vol. 33, no. 5, pp. 60–67, 2016.
- [6] L. Bessey, et al., "SRE: A case study on availability," in *Proc. 15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2016.
- [7] P. Yin, M. Tomic, and R. K. L. Loo, "A reference model of financial processes," Bank for International Settlements, Monetary and Economic Dept., Basel, Switzerland, Working Paper No. 14-R, 2007.
- [8] D. Dzhemerosh, S. Dragoi, V. Georgescu, A. Jezequel, and J. Ychs, "Scaling static analyses: A study of 26 code analyzers," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2018, pp. 405–429.
- [9] H. Pearce, B. Tan, S. Sarma, A. Flammen, and D. Kumar, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2022, pp. 1041–1054.
- [10] S. Peng, E. Kalliamvakou, P. Zhou, et al., "The impact of AI on developer productivity: Evidence from GitHub Copilot," arXiv:2302.06590, 2023.
- [11] M. Chen, J. Tworek, H. Jun, et al., "Evaluating large language models trained on code," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021, vol. 34.
- [12] C. E. Jimenez, S. Pugh, A. S. M. et al., "SWE-bench: Can language models resolve real-world GitHub issues?" in *Int. Conf. Learning Representations (ICLR)*, 2024.

- [13] A. Anthropic, “Model Context Protocol,” specification and announcement, 2024. [Online]. Available: <https://www.anthropic.com/news/model-context-protocol>
- [14] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2002.
- [15] S. Newman, *Building Microservices*, 2nd ed. Sebastopol, CA: O’Reilly Media, 2021.
- [16] F. M. Brooks, “No silver bullet,” in *Proc. AFIPS 1986 National Computer Conference*, 1986, pp. 236–248.